LLM-Based Network Troubleshooting Agent

COMS6200 - Project Proposal / Progress Report (Part A)

The University of Queensland

[TEAM TO INSERT TEAM MEMBER 1 + STUDENT ID]

[TEAM TO INSERT TEAM MEMBER 2 + STUDENT ID]

[TEAM TO INSERT TEAM MEMBER 3 + STUDENT ID]

[TEAM TO INSERT TEAM MEMBER 4 + STUDENT ID]

Submission deadline: 4 September 2026, 3:00 pm

Final project report deadline: 16 October 2026, 3:00 pm

Abstract

This project investigates whether a tool-using large language model (LLM) can diagnose and localise common faults in an OpenFlow-controlled software-defined network more accurately or efficiently than a deterministic troubleshooting script. The testbed uses Mininet, Open vSwitch, OpenFlow 1.3 and the ONOS SDN controller in a six-switch, eight-host Layer-2 topology with redundant paths. Five reproducible fault scenarios are proposed: link failure, congestion, ACL/policy error, forwarding loop, and a missing or incorrect OpenFlow rule. The LLM will not receive unrestricted shell access or hidden fault labels; instead, it will gather evidence through a small set of structured diagnostic functions for topology, ping, port state/statistics, flow tables and throughput. A planned scripted troubleshooting baseline will use the same underlying information. Evaluation will focus on diagnosis accuracy, fault localisation, tool calls, diagnosis time and, for one controlled extension, repair success. The scope intentionally prioritises a complete, repeatable experimental pipeline over a large topology or broad fault catalogue. Current progress demonstrates the enhanced six-switch/eight-host topology, a reproducible ONOS 2.7.0 installation, OpenFlow 1.3 controller integration, deterministic loop-free baseline forwarding with successful host connectivity (`pingall` 0% dropped, 56/56), and a runtime-verified structured telemetry gateway. Automated fault injection and LLM integration remain pending.

1. Introduction

Network troubleshooting is often an iterative process: an operator observes a symptom, selects a diagnostic test, interprets the result, narrows the candidate causes and repeats until a root cause can be localised. Software-Defined Networking (SDN) makes this process especially suitable for automation because the control plane can maintain a logically centralised view of topology and forwarding state while OpenFlow exposes programmable match-action rules and counters. This project focuses on an LLM-based troubleshooting agent that can use topology information, ping, flow tables and link/port statistics to diagnose faults such as congestion, link failures, ACL errors and forwarding loops.

Large language models add a different capability from conventional rule-based monitoring: they can choose diagnostic actions adaptively, interpret heterogeneous evidence and produce natural-language explanations. However, a useful university project should not assume that this flexibility automatically produces correct diagnoses. Recent network-agent research reports that larger LLM-based agents succeed more often in detecting network issues but still struggle to localise faults and identify root causes [1], while work on symptom-aware, phase-gated diagnostic escalation shows that structuring evidence acquisition and hypothesis commitment can improve root-cause identification [2]. This motivates an empirical comparison rather than a demonstration-only system.

The primary research question is therefore: How accurately and efficiently can a tool-using LLM agent diagnose and localise common faults in an OpenFlow-controlled Mininet network compared with a deterministic scripted troubleshooting baseline? A secondary question will be retained only as a controlled extension: Can a diagnosed link failure be safely resolved through a constrained OpenFlow rerouting action and validated without creating unintended connectivity changes?

The project is intentionally scoped around one representative topology, five fault scenarios, a bounded diagnostic tool set and a paired evaluation against one credible scripted troubleshooting baseline. This is consistent with course-coordinator guidance received by the team: there is no fixed topology size or fault count; the network should start minimal and scale only as required by the scenarios; automatic repair is optional; and API usage should be instrumented and capped. The expected contribution is not a claim of a novel general-purpose network copilot. Instead, the project will provide a controlled and reproducible SDN testbed in which an LLM and a scripted method diagnose the same hidden faults from the same observable evidence.

2. Background

2.1 Software-Defined Networking and OpenFlow

SDN separates network-control logic from packet forwarding. In the classic architecture, a logically centralised controller maintains network state and programs data-plane switches through a southbound interface. OpenFlow was proposed around a standardised interface that allows a controller to add and remove entries in a

switch's flow table [3]. This separation is central to the proposed project: faults can exist in the physical/link state, in controller knowledge, or in the forwarding policy installed in Open vSwitch, and the troubleshooting agent must use evidence across these layers rather than treat every failure as a simple connectivity problem.

An OpenFlow flow entry implements a match-action abstraction. Packets can be matched on fields such as ingress port, MAC address, EtherType, IP addresses and transport ports, then forwarded to one or more output ports, sent to the controller or dropped. OpenFlow also maintains per-flow and per-port counters. OpenFlow's match-and-action primitives, including selective forwarding and dropping, provide mechanisms that this project can use to construct controlled ACL errors, blackholes and forwarding-loop scenarios [3], [4]. OpenFlow 1.3 is used because it provides the flow-programming, controller-messaging and statistics functionality required by the proposed experiments [4], is supported by Open vSwitch [5] and by the ONOS OpenFlow provider [6], and aligns with COMS6200 course material. Features introduced in later OpenFlow versions are not required for the project's fault scenarios or evaluation.

Relevant controller messages include Packet_In, Packet_Out and Flow_Mod. A table miss can cause a switch to encapsulate a packet in Packet_In for the controller; the controller can issue Packet_Out for an individual packet or Flow_Mod to install persistent forwarding behaviour. Flow and PORT_STATS requests provide counters that can be used for traffic and fault diagnosis. The project uses these concepts directly: ONOS has installed loop-free baseline forwarding; the structured telemetry gateway queries state and counters; and planned fault injectors will create specific incorrect rules or link conditions.

2.2 Mininet, Open vSwitch and topology knowledge

Mininet provides a practical experimental platform by using lightweight OS-level virtualisation to create virtual hosts, switches, controllers and links on a single machine while allowing real network software to run in the emulated environment [7]. Custom Python topologies and traffic-control links allow experiments to be automatically recreated. Open vSwitch (OVS) provides the software data plane controlled through OpenFlow [5]. The experiment harness forces OpenFlow 1.3 and avoids uncontrolled bridge fallback in the redundant topology, because cyclic Layer-2 connectivity must be governed by explicit loop-free forwarding rules.

A controller must know the network topology before it can reason about expected paths. Pakzad et al. describe topology discovery as a critical controller service and detail the de facto LLDP-based approach in which controllers send LLDP packets through OpenFlow Packet_Out messages and receive them through Packet_In messages to infer inter-switch links [8]. They evaluate topology discovery across multiple network topologies using the Mininet network emulator and additionally validate their results on the OFELIA SDN testbed [8]. This project does not propose a new discovery protocol. Future diagnostic tools will consume ONOS device and inter-switch link state, and they will obtain host attachments from the configured Mininet topology rather than from dynamic host discovery.

2.3 Network telemetry and LLM tool use

The troubleshooting system will combine active tests with controller state. Active tests such as ICMP ping reveal reachability, loss and round-trip delay; short iperf tests provide a direct throughput signal. Controller and OVS state provide port/link status, flow tables and counters. For a Layer-2 fabric, ordinary IP traceroute is not expected to reveal each internal OpenFlow switch because the switches are not IP router hops. The project will therefore not redesign the network as Layer 3 simply to make traceroute useful. Instead, the equivalent path evidence will be derived from topology, expected path computation, flow-table output actions and port statistics.

The LLM will interact with this evidence through structured function calls rather than an unrestricted terminal. The general design follows the tool-using agent pattern exemplified by ReAct, in which a language model interleaves reasoning traces with task-specific actions and observations from an external environment [9]. OpenAI documents GPT-5 mini as supporting function calling and structured outputs and lists the version-pinned snapshot gpt-5-mini-2025-08-07 [10]. Given the bounded experiment size, GPT-5 mini is proposed as the single primary model; token usage and API cost will be logged for each episode. Model benchmarking is deliberately out of scope because COMS6200 has a separate project idea devoted to comparing language models.

3. Related Work

Foundational SDN work established the programmable-control model used in this project. McKeown et al. proposed OpenFlow as a standardised interface to switch flow tables [3], while Lantz, Heller and McKeown showed that Mininet could rapidly prototype software-defined networks on a single machine [7]. Pakzad et al. evaluated the control overhead of the de facto OpenFlow topology-discovery approach and proposed modifications that reduce controller and control-traffic overhead while preserving discovery functionality [8]. These works provide the network-control foundation, but they do not evaluate LLM-based diagnostic decision making.

More recent LLM-agent literature is directly relevant to the diagnostic architecture. ReAct introduced a framework that interleaves language-model reasoning with task-specific actions and external observations [9]. NIKA provides a public benchmark for LLM-driven network incident diagnosis and troubleshooting, and its evaluation shows that LLM agents still struggle with fault localisation and root-cause identification [1]. SADE introduces a symptom-aware, phase-gated diagnostic workflow and reports substantially improved root-cause performance compared with a ReAct-based baseline [2]. Nissist uses LLMs to extract knowledge from unstructured troubleshooting guides and historical incident-mitigation information to support incident mitigation [11]. NIKA, SADE and Nissist are useful context but are not identical to the proposed experiment: this project focuses on a small, fully controlled OpenFlow testbed in which the injected fault, topology, diagnostics and scoring are known and reproducible.

Industry products also demonstrate practical interest in AI-assisted network operations. Cisco product documentation describes AgenticOps capabilities in which the Cisco AI Assistant can diagnose root causes and execute automated, deterministic fixes for network issues [12]. Juniper product documentation describes Marvis as an AI-driven, interactive virtual network assistant with real-time network visibility that can identify issues, recommend corrective actions and, in self-driving mode, automatically fix selected issues [13]. These products establish industry relevance, but their internal data, policies and evaluation procedures are proprietary. The proposed COMS6200 testbed therefore complements commercial systems by enabling transparent ground truth, reproducible fault injection and a fair comparison with a scripted troubleshooting baseline.

The project will avoid overclaiming novelty. Its academically defensible contribution is a small experimental framework that evaluates tool selection, diagnosis and localisation under controlled SDN faults, with the same observable data available to the LLM and the planned scripted troubleshooting baseline. This directly addresses the course project question while remaining feasible within the remaining semester.

4. Project Aim and Scope

4.1 Aim and objectives

The project aim is to implement and evaluate a bounded LLM troubleshooting agent for an OpenFlow-controlled Mininet network. The system must diagnose what failed, localise where the fault resides, cite the evidence it used, and be compared with a deterministic troubleshooting procedure. The implementation will emphasise experimental control and reproducibility rather than topology size.

- Implement one six-switch/eight-host redundant Layer-2 Mininet topology controlled through OpenFlow 1.3.
- Create five automated, resettable fault scenarios covering the four official categories plus one low-cost forwarding-rule fault.
- Expose a small set of structured, read-only diagnostic tools to one OpenAI model.
- Implement one deterministic scripted troubleshooting baseline using the same underlying information.
- Run paired repeated trials and measure diagnosis accuracy, localisation, diagnostic effort and time.
- If the core diagnosis experiment is complete, demonstrate one allow-listed automatic reroute for a link failure and validate the result.

4.2 In scope and out of scope

In scope are fault classification, precise localisation, root-cause explanation, bounded tool selection, repair recommendation and optional single-case repair. The primary network will remain Layer 2. Fault injection will cover link state, traffic contention, policy/ACL state and OpenFlow forwarding state. The agent will operate only through predefined functions and will not receive the fault label or injector configuration.

Out of scope are multi-topology scaling experiments, compound faults, controller-failure benchmarking, duplicate-IP or host-addressing faults, asymmetric routing, full Layer-3 routing, a large model-comparison study and broad autonomous remediation. These are reasonable future extensions but would dilute the core experiment under the 16 October deadline. ONOS 2.7.0 is the selected controller for the experimental testbed [6]. It was chosen for course alignment, verified OpenFlow 1.3 operation on the team server, a global topology/flow view, and a localhost REST API that keeps the LLM tool layer separate from the controller. Controller comparison is outside the project scope.

5. Methodology

5.1 Experimental topology and SDN environment

The primary testbed contains six OpenFlow switches, eight hosts, 10 physical inter-switch links and 9 physical host-switch attachments (19 links in total). Switches s1 and s2 form redundant cores. Access switches s3-s6 each connect to both cores. The s1-s2 inter-core link is retained as an alternate/experimental path and does not carry normal traffic. An additional s3-s4 access-edge test link is included so a narrow selected flow can be steered through a short edge/core cycle or an unexpected core-bypass path without enlarging the node set. Hosts attach as h1-h2 to s3, h3-h4 to s4, h5 and the primary h6 interface to s5, and h7-h8 to s6. Host h6 also has a secondary physical attachment to s6, reserved for an optional later multihoming/failover experiment; it is not used in baseline forwarding and is not one of the five mandatory faults. These extra paths increase diagnostic diversity rather than topology size for its own sake. The design is smaller than a Clos or fat-tree but still supports redundancy, alternate paths, cycles and multiple fault locations, while keeping flow tables and diagnosis traces interpretable.

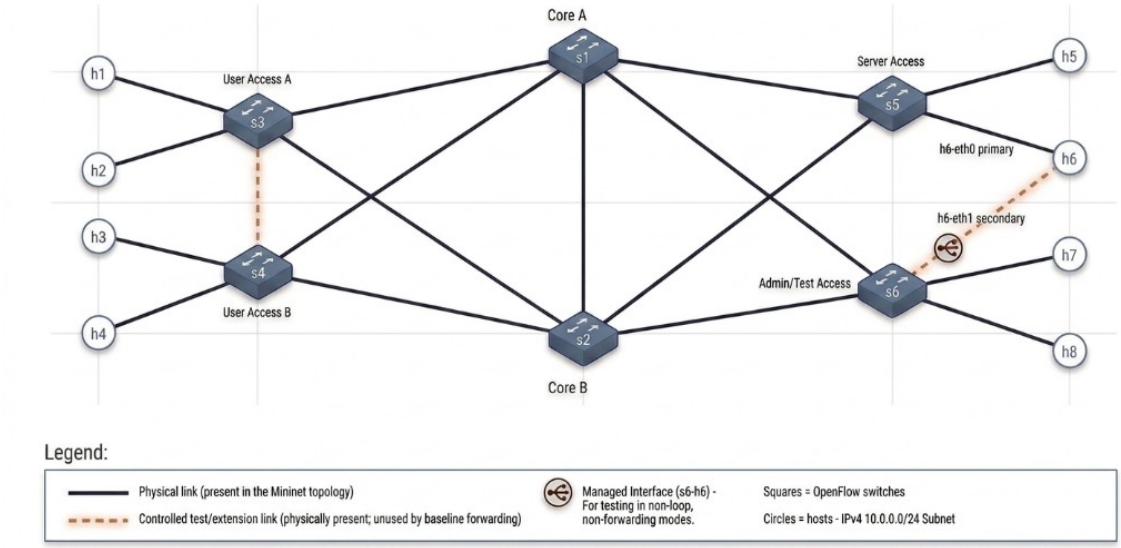


Figure 1. Enhanced six-switch/eight-host dual-core Mininet topology with an access-edge test link and one optional multihomed host.

All hosts share a single IPv4 subnet (10.0.0.0/24) to minimise routing implementation overhead; h6's secondary interface has no baseline IPv4 address. The verified baseline installs explicit destination-MAC forwarding through s1 as the active core (s3/s4/s5/s6 → s1). Broadcast ARP requests use ingress-specific ARP-tree rules on active-tree ports only; ARP replies follow the same destination-MAC forwarding. Baseline forwarding does not use the s2 access uplinks, s1-s2, s3-s4 or h6-s6 secondary attachment, so the physical cycles remain loop-free in the normal case. The topology is implemented in Python using Mininet and TCLink so that bandwidth and delay can be controlled where required. OVS bridges are configured for OpenFlow 1.3 and a secure controller fail mode. ONOS 2.7.0 supplies topology and OpenFlow state through its OpenFlow provider, LLDP link

discovery and localhost REST API [6]; version and dependency information are recorded in the experiment log.

5.2 Agent architecture and diagnostic tools

The operator fault injector applies controlled faults to Mininet/OVS (and, where required, temporary ONOS experiment flows) and retains hidden ground truth outside the diagnostic path. Fault Catalogue v1 (F1–F5) is implemented and runtime-verified. Mininet/OVS is programmed by ONOS 2.7.0 over OpenFlow 1.3. ONOS exposes controller state through a localhost read-only REST interface. A telemetry/tool gateway normalises that state into the bounded tools in Table 1. Both the OpenAI LLM agent and the planned scripted troubleshooting baseline will call only that same gateway and therefore receive equivalent diagnostic information. Neither method will receive ONOS credentials, unrestricted REST access, fault labels or injector configuration. Both will return a compatible structured diagnosis to the evaluator/logger. Hidden ground truth remains evaluator-only and is never shown to either diagnostic method. This prevents direct leakage such as showing the LLM a tc configuration that literally states "loss 10%". The six tools and Fault Catalogue v1 were runtime-verified on the Ubuntu reference server on 27 August 2026; the scripted troubleshooting baseline and LLM agent are not yet implemented.

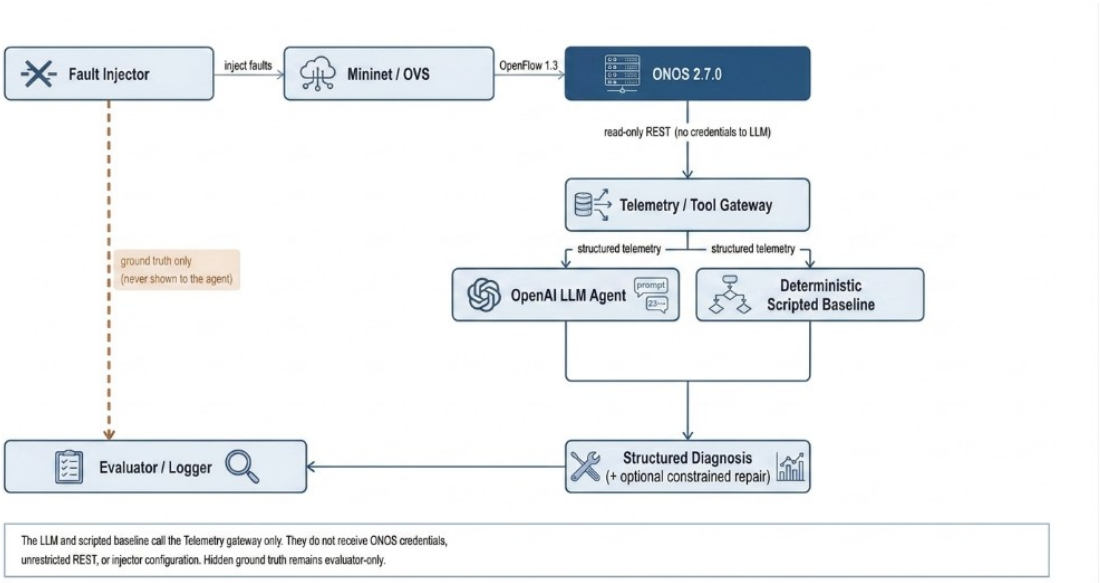


Figure 2. Proposed troubleshooting architecture. Ground truth is isolated from the diagnostic agent; the LLM reaches ONOS only through a read-only telemetry gateway.

The primary LLM will use the OpenAI API with structured function calling. The version-pinned GPT-5 mini snapshot gpt-5-mini-2025-08-07 is proposed because the official model documentation lists support for function calling and structured outputs and explains that snapshots lock a specific model version for consistent behaviour [10]. Each episode will log the exact model identifier, prompt/version, token usage, tool calls and wall-clock time. API budgets and per-episode limits will be enforced to prevent runaway token use.

Tool	Purpose	Compact return
get_topology()	Combined topology from ONOS device/link state and the configured host map	switches, links, ports/status, configured host attachments
ping(src,dst)	Reachability, packet loss and RTT	sent/received, loss %, min/avg/max RTT
get_port_status(sw)	Administrative/operational port state	port, peer, up/down
get_port_stats(sw)	Traffic counters and derived link rates	packet/byte deltas, drops/errors, utilisation
get_flow_table(sw)	Inspect OpenFlow matches, priorities, actions and counters	normalised flow entries
iperf(src,dst)	Short TCP throughput test for congestion/performance faults	duration, transfer, Mbit/s

Table 1. Controlled diagnostic functions exposed to the LLM and the planned scripted troubleshooting baseline.

The implemented get_topology() tool combines two sources. ONOS REST supplies OpenFlow devices, inter-switch links, device ports and controller-visible status. The deterministic Mininet topology specification supplies configured host attachments, host-to-switch port mapping, host identities, and h6's primary and secondary attachments. This hybrid design is required because ONOS dynamic host discovery is not a reliable source of host locations under the verified baseline: after `pingall` succeeded 56/56, REST `/hosts` still returned HTTP 200 with host_count 0. That result is expected. The explicit ARP tree forwards ARP in the data plane, so hostprovider does not necessarily receive the Packet_In traffic needed to learn attachments. The project does not change ARP behaviour or enable reactive forwarding merely to populate `/hosts`.

Traceroute is not included as a core tool because the testbed is an L2 OpenFlow fabric; ordinary IP traceroute would typically show only the endpoint path at Layer 3 and not reveal each internal switch. If retained for completeness, it will be documented as secondary evidence only. The more relevant SDN path evidence is the combination of topology, expected forwarding graph, flow output actions and counter activity. For the optional h6 multihoming extension only, limited host-interface state may later be added as an extra tool; it is not part of the mandatory LLM interface.

5.3 Fault scenarios and injection

Five scenarios will be implemented. Each scenario will be declared in a small configuration file and injected by Python so it can be repeated and reset from a known healthy forwarding state. The project will not implement dozens of variants. Instead, every scenario will have one clearly defined expected root cause and a measurable symptom, and the same scenario definition can be repeated with controlled seeds or traffic timing where relevant.

Scenario	Injected fault	Mechanism	Expected diagnostic evidence	Correct outcome
F1 Link failure	Disable one primary access-to-core link (e.g., s3-s1)	Mininet link status / port down	Path failure or fallback path; link state confirms location	Locate failed link; recommend alternate path; optional reroute demo
F2 Congestion	Competing flows share a constrained core uplink	TCLink capacity + background iperf traffic	Ports remain UP; high utilisation; throughput/RTT degrade	Identify congested link and competing load
F3 ACL/policy error	High-priority drop rule blocks one source/service	OpenFlow flow rule via the controlled fault-injection layer / ONOS interface	Selective protocol/source failure; healthy links	Locate conflicting drop rule and correct/delete it
F4 Forwarding loop	Narrow test flow is sent around a short edge/core cycle using s3-s4	High-priority output rules with hard timeout	No delivery; abnormal counter growth on the s3-s4 cycle	Identify cyclic rules and remove offending entry
F5 Missing/incorrect flow	Delete required flow or send to wrong egress	Flow delete/override	Topology healthy but target flow blackholes/misroutes	Locate missing/wrong rule and restore expected action

Table 2. Core fault scenarios. The first four correspond directly to the official project categories.

Forwarding-loop injection will be safety constrained. Only a narrowly matched test flow will be affected, using the dedicated s3-s4 access-edge path so the cycle can be formed without destabilising all traffic. Rules will use a short hard timeout, traffic will be low rate, and a watchdog will remove fault-cookie rules if the scenario does not terminate as expected. Exact flow-rule syntax will be defined when the injector is implemented. For congestion, the experiment will distinguish a true load-induced condition from a static low-bandwidth link: the correct diagnosis should rely on simultaneous utilisation/counter evidence and competing traffic rather than merely noticing a configured bandwidth value.

The h6-s6 secondary attachment is an optional advanced extension if the core five-scenario experiment finishes early. It is not a sixth mandatory fault. A possible later question is whether the agent can distinguish host-side failover or interface state from stale controller/flow-table state. No automatic host failover is implemented in Part A.

5.4 Deterministic scripted troubleshooting baseline

The planned scripted troubleshooting baseline will be a credible deterministic troubleshooter rather than a deliberately weak comparator. It will use the same telemetry/tool gateway and the same episode budget as the LLM. A simple decision procedure is sufficient: first test endpoint reachability; then inspect relevant port/link state; if the path remains physically healthy, collect short performance/counter measurements; then inspect flow-table state for drops, priority conflicts, missing rules, wrong outputs or cycles. The scripted troubleshooting baseline will return the same structured diagnosis fields as the LLM. Thresholds for congestion or abnormal loss will be calibrated on healthy pilot runs and frozen before the final experiment. This comparison method is distinct from the already implemented `org.coms6200.baseline` forwarding application.

Fairness requires that neither method receives hidden ground truth and that both operate on a clean topology with equivalent diagnostic information from the same gateway. The LLM may choose tools adaptively; the scripted troubleshooting baseline follows its fixed procedure. This difference is the intended experimental variable.

5.5 Ground truth, experiment size and evaluation

Every fault injector will record an evaluator-only ground-truth object containing fault family, location, affected traffic and expected corrective action. This object will never be passed into the LLM prompt or the scripted troubleshooting baseline. Before each episode the topology will be reset or rebuilt, normal connectivity will be verified, and the fault will then be injected. After the diagnostic run, the evaluator will compare the predicted family and location against the hidden record.

The proposed final experiment contains five scenarios with eight repetitions per scenario, giving 40 fault instances for each method. Each instance will be run once with the LLM and once with the scripted troubleshooting baseline from a fresh network state, for approximately 80 diagnostic runs in total. Eight repetitions are sufficient to reveal obvious instability in the LLM without consuming the semester on a large benchmark. For stochastic congestion or packet timing, scenario seeds and realised measurements will be logged. If pilot results show extremely low variance, the team may keep eight repetitions; if variance is high and time permits, the upper limit will be ten.

Metric	Definition	Role
Diagnosis accuracy	Percentage of episodes with the correct fault family	Primary effectiveness metric
Localisation accuracy	Correct switch/link/flow-rule location	Separates generic recognition from precise diagnosis
Tool calls	Number of diagnostic function calls before final answer	Diagnostic efficiency
Diagnosis time	Wall-clock time from symptom to final diagnosis	Operational efficiency
Repair success	For optional F1 repair only: connectivity restored and invariants preserved	Controlled extension
Token usage / cost	Input/output tokens and estimated API cost	Operational record; secondary metric
Valid tool-call rate	Malformed/invalid calls divided by attempted calls	Reliability check

Table 3. Evaluation metrics retained for the final experiment.

Results will be reported as raw counts, percentages and simple descriptive statistics. Accuracy will be broken down by fault scenario. Tool calls and diagnosis time will be summarised with median and range (or interquartile range if appropriate), because API/network timings can be skewed. Confidence intervals may be reported for diagnosis accuracy where useful. Sophisticated statistical testing is not required unless the completed dataset justifies it; a clear paired comparison and transparent raw results are more important than adding unnecessary analysis.

6. Project Plan

The schedule is constrained by the Part A deadline of 4 September 2026, 3:00 pm, and the final report deadline of 16 October 2026, 3:00 pm. Environment setup, the enhanced main topology, ONOS 2.7.0/OpenFlow 1.3 integration, deterministic loop-free baseline forwarding, the structured telemetry gateway and the frozen Fault Catalogue v1 (F1–F5 inject/reset) are already complete. The next milestone is the first LLM tool-calling diagnosis (M6) using OpenAI via Cloudflare AI Gateway against this frozen catalogue and M4 telemetry. The scripted troubleshooting baseline and repeated experiments follow after that vertical slice. This ordering prioritises the 30% "Progress So Far" component of the Part A rubric and reduces the risk of discovering integration problems late in the semester.



Figure 3. Calendar-based implementation and evaluation plan, 18 August–16 October 2026.

Dates	Milestone / deliverable	Status
18–20 Aug	Environment bootstrap and initial Mininet capability validation	Completed
20–24 Aug	Enhanced `main6s8h` topology, ONOS 2.7.0 and OpenFlow 1.3 integration	Completed
20–27 Aug	Deterministic baseline forwarding and structured telemetry gateway	Completed
24–27 Aug	Fault Catalogue v1: F1–F5 automated injection, observation and clean reset	Completed
28 Aug–4 Sep	Part A refinement, team review, final validation and submission	In progress
5–11 Sep	M6: OpenAI + Cloudflare AI Gateway integration and first LLM tool-calling diagnosis	Planned
12–18 Sep	Deterministic scripted troubleshooting baseline and initial pilot	Planned
19–25 Sep	Pilot calibration and freeze final experimental protocol	Planned
26 Sep–7 Oct	Repeated paired LLM vs scripted-baseline experiments	Planned
8–12 Oct	Results analysis and optional one-case repair demonstration if core work is complete	Planned
13–16 Oct	Final report, demo preparation and submission	Planned

Table 4. Key milestones and deliverables.

7. Progress So Far

The team has completed practical Mininet work, verified ONOS 2.7.0/OpenFlow 1.3 integration, installed deterministic loop-free baseline forwarding, runtime-verified a bounded structured telemetry gateway, and runtime-verified the complete Fault Catalogue v1 (F1–F5) with clean reset on the team Ubuntu server. Early Mininet/TCLink experiments used OVS bridge fallback for capability checks; the project topology now runs against ONOS with OpenFlow 1.3, secure fail mode and explicit s1-core host forwarding. The report still distinguishes confirmed results from tasks that require further evidence, in particular LLM diagnosis and the

scripted troubleshooting baseline.

Work item	Status	Evidence
Mininet parameterised topologies	Completed	Linear and tree topologies created; host/switch interfaces inspected with net.
Custom Python topology	Completed	Repository topology progress5s2h reproduced the five-switch/two-host layout; nodes, net and links confirmed all seven nodes and six intended links, with all links reporting OK.
Connectivity validation	Completed	Fresh h1-h2 test: 10/10 ping replies, 0% loss; RTT min/avg/max 0.103/0.193/0.873 ms.
Bandwidth control	Completed	Fresh progressbw10 test: a 10 Mbit/s TCLink delivered 9.38 Mbit/s receiver throughput over a 5 s iperf3 run. An impaired 1 Mbit/s variant later delivered 832 Kbit/s under added delay/loss.
Delay control	Completed	Fresh progressimpaired test: 5 ms configured delay per direction produced 10.171 ms average RTT over a 100-packet h1-h2 ping (min/max 10.115/10.690 ms).
Packet-loss control	Completed	Fresh progressimpaired test: 10% loss configured on each direction of the affected TCLink produced 81/100 ping replies (19% round-trip loss), consistent with the bidirectional impairment.
ONOS + OpenFlow 1.3 main topology	Completed	Enhanced main6s8h connected all six OVS switches to ONOS 2.7.0 using OpenFlow 1.3 and secure fail mode; ONOS discovered all six deterministic DPIDs and 20 directed links representing 10 physical inter-switch links, including s3-s4 at ports 5/5. Host h6 had h6-eth0 on s5:4 (10.0.0.6/24) and h6-eth1 on s6:5 (no baseline IPv4). REST /devices, /links, /flows and /topology returned HTTP 200.
Deterministic loop-free forwarding	Completed	Explicit ONOS-owned `org.coms6200.baseline` flow rules installed dest-MAC unicast (priority 41000) and ingress-specific ARP-tree forwarding (priority 42000) through s1, leaving s2, s1-s2, s3-s4 and h6-s6 outside normal forwarding. 56/56 baseline flows reached ADDED (s1:12, s2:0, s3–s6:11 each) beside 18 controller punt flows. Mininet `pingall` recorded 0% dropped (56/56 received) with `org.onosproject.fwd` inactive and no NORMAL/STP/FLOOD.
Structured topology/flow/port telemetry	Completed	Six bounded tools (`get_topology`, `ping`, `get_port_status`, `get_port_stats`, `get_flow_table`, `iperf`) returned normalised JSON on live `main6s8h`+ONOS+M3. `verify_telemetry.sh` passed; pingall remained 56/56. Evidence: `evidence/progress/telemetry/structured-telemetry-2026-08-27.txt`.
Automated project fault injector	Completed	Fault Catalogue v1 frozen on the Ubuntu reference testbed (`verify_fault_catalogue.sh` ALL FIVE PASSED). F1 primary s3–s1 link down (20→18 directed links; h1→h5 fails). F2 TBF congestion on s1–s5 with background load (reachable but throughput □ healthy). F3 selective DROP h1→h5 on s3 via neutral `org.coms6200.experiment`. F4 bounded unicast loop s3→s4→s1→s3 with safety TBF. F5 missing s3 baseline unicast for h5 (55 ADDED then restored to 56). Final pingall 56/56. Evidence: `evidence/progress/faults/fault-catalogue-v1-2026-08-27.txt`.
OpenAI function-calling diagnosis	Not yet evidenced	[TEAM TO INSERT FIRST TOOL-CALL TRACE AND TOKEN USAGE HERE]

Table 5. Confirmed progress and evidence gaps as of 27 August 2026.

The 100-packet impaired-link run observed 19% end-to-end ICMP loss, consistent with independent 10% loss affecting both request and reply directions, while the same configuration reduced TCP receiver throughput to 832 Kbit/s with 77 retransmissions.

The performance measurements above are not final project results and should not be presented as evidence that the LLM agent already works. Their value is methodological: they confirm that the team can create topologies, inspect interfaces, measure throughput/latency and apply controlled TCLink impairments, which are required for the planned congestion and link-quality experiments. Controller/OpenFlow switch discovery, deterministic host forwarding, structured telemetry and the frozen Fault Catalogue v1 are now demonstrated. The next implementation milestone is LLM/API integration (OpenAI via Cloudflare AI Gateway) and the first tool-calling diagnosis episode against this catalogue.

Enhanced main6s8h / ONOS 2.7.0 runtime evidence (20 August 2026): 6/6 devices `of:0000000000000001`–`of:0000000000000006` available with `protocol=OF_13`; topology `{devices:6, links:20, clusters:1}`; s3:5↔s4:5 ACTIVE in both directions; 18 controller punt flows (3 per switch); 56 `org.coms6200.baseline` flows ADDED; `pingall` 0% dropped (56/56); `org.onosproject.fwd` not active. ONOS REST `/hosts` returned HTTP 200 with host_count 0 after that successful `pingall`; this is expected under data-plane ARP forwarding and is not treated as a connectivity failure.

Verified telemetry example (Ubuntu, 27 August 2026), ping h1→h5:

```json
{"tool": "ping", "ok": true, "data": {"sent": 3, "received": 3, "loss_pct": 0.0,
"src": "h1", "dst": "h5", "dst_ip": "10.0.0.5"}, "error": null}
```

`get_topology()` summary on the same healthy baseline: 6 switches, 10 physical inter-switch links, 9 configured host attachments, `onos_host_discovery=not_used`. Fault injection and LLM diagnosis remain placeholders below.

8. Risks and Mitigation

Risk	Consequence	Mitigation
ONOS resource/container dependency	Controller instability or memory pressure on the small team server could interrupt experiments.	Use version-pinned ONOS 2.7.0, tested JVM/container memory limits, localhost-only OpenFlow/REST/CLI bindings, environment verification and repeatable restart/cleanup. [6]
OpenFlow / L2 loop safety	A deliberately injected loop can create a packet storm.	Match only one test flow, short hard_timeout, low-rate probe, watchdog cleanup and full network reset.
LLM invalid or excessive tool use	Malformed calls, repeated diagnostics or token growth.	Strict schemas/enums, 8-10 call cap, output normalisation, per-episode token/time limits.
Insufficient implementation time	Too many fault variants reduce quality or prevent evaluation.	Keep one topology, five scenarios and one model. Cut optional repair before cutting diagnosis/evaluation.
Experimental contamination	Residual flows, tc settings, traffic processes or ARP state alter later trials.	Automated clean-state checks; restart/rebuild topology between final episodes where practical.
API variability/cost	Non-deterministic answers or unexpected usage.	Use a version-pinned model snapshot, log tokens, run 8 paired repetitions, set hard usage limits and keep prompts/tool outputs concise.

Table 6. Principal project risks and mitigations.

9. Conclusion

The proposed project is a deliberately constrained but technically substantive evaluation of LLM-assisted SDN troubleshooting. A six-switch/eight-host dual-core Mininet topology supplies redundant paths and enough forwarding complexity to reproduce the four official fault categories, while a fifth missing/incorrect-flow scenario adds useful OpenFlow-specific coverage with little extra implementation cost. The LLM will gather evidence through a small, auditable tool set and will be compared fairly with a planned scripted troubleshooting baseline using hidden ground truth and repeated paired trials.

This scope is achievable by 16 October because it avoids unnecessary Layer-3 routing, topology scaling, compound faults and broad autonomous remediation. The immediate priority is one automated/resettable fault and one LLM tool-calling episode. If that foundation is demonstrated and the final five scenarios are executed reproducibly, the project will provide clear evidence of technical understanding, initiative, measurable methodology and genuine implementation progress aligned with the COMS6200 marking criteria.

References

[1] Z. Wang, A. Cornacchia, A. Sacco, F. Galante, M. Canini, and D. Jiang, "A Network Arena for Benchmarking AI Agents on Network Troubleshooting," arXiv:2512.16381, 2025 [Online]. Available: https://arxiv.org/abs/2512.16381.

[2] K.-H. Tseng, N. Bogahawatta, Y. Ginige, K. Dekic, A. Sivanathan, and S. Seneviratne, "SADE: Symptom-Aware Diagnostic Escalation for LLM-Based Network Troubleshooting," arXiv:2605.04530, 2026 [Online]. Available: https://arxiv.org/abs/2605.04530.

[3] N. McKeown, T. Anderson, H. Balakrishnan, G. Parulkar, L. Peterson, J. Rexford, S. Shenker, and J. Turner, "OpenFlow: Enabling Innovation in Campus Networks," ACM SIGCOMM Computer Communication Review, vol. 38, no. 2, pp. 69-74, 2008, doi: 10.1145/1355734.1355746. [Online]. Available: https://doi.org/10.1145/1355734.1355746.

[4] Open Networking Foundation, "OpenFlow Switch Specification Version 1.3.5," ONF TS-023, Mar. 2015. [Online]. Available: https://opennetworking.org/wp-content/uploads/2014/10/openflow-switch-v1.3.5.pdf.

[5] Open vSwitch, "Using OpenFlow," Open vSwitch Documentation, accessed Aug. 18, 2026. [Online]. Available: https://docs.openvswitch.org/en/stable/faq/openflow/.

[6] Open Networking Foundation, "ONOS 2.7.0: Open Network Operating System," ONOS Wiki, accessed Aug. 20, 2026, 2021. [Online]. Available: https://wiki.onosproject.org/display/ONOS/ONOS.

[7] B. Lantz, B. Heller, and N. McKeown, "A Network in a Laptop: Rapid Prototyping for Software-Defined Networks," in Proceedings of the 9th ACM SIGCOMM Workshop on Hot Topics in Networks, Monterey, CA, USA, 2010, doi: 10.1145/1868447.1868466. [Online]. Available: https://doi.org/10.1145/1868447.1868466.

[8] F. Pakzad, M. Portmann, W. L. Tan, and J. Indulska, "Efficient topology discovery in OpenFlow-based Software Defined Networks," Computer Communications, vol. 77, pp. 52-61, 2016, doi: 10.1016/j.comcom.2015.09.013. [Online]. Available: https://doi.org/10.1016/j.comcom.2015.09.013.

[9] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. R. Narasimhan, and Y. Cao, "ReAct: Synergizing Reasoning and Acting in Language Models," in International Conference on Learning Representations, 2023. [Online]. Available: https://iclr.cc/virtual/2023/poster/11003.

[10] OpenAI, "GPT-5 mini Model," OpenAI API Documentation, accessed Aug. 18, 2026. [Online]. Available: https://developers.openai.com/api/docs/models/gpt-5-mini.

[11] K. An, F. Yang, J. Lu, L. Li, Z. Ren, H. Huang, L. Wang, P. Zhao, Y. Kang, H. Ding, Q. Lin, S. Rajmohan, D. Zhang, and Q. Zhang, "Nissist: An Incident Mitigation Copilot based on Troubleshooting Guides," in ECAI 2024, 2024, doi: 10.3233/FAIA241032. [Online]. Available: https://doi.org/10.3233/FAIA241032.

[12] Cisco, "AgenticOps for Enterprise Networking," Cisco Product Documentation, accessed Aug. 18, 2026. [Online]. Available: https://www.cisco.com/site/us/en/solutions/artificial-intelligence/agentic-ops/net-ops/index.html.

[13] Juniper Networks, "Troubleshoot with Marvis," Juniper Mist Documentation, accessed Aug. 18, 2026. [Online]. Available: https://www.juniper.net/documentation/us/en/software/mist/mist-wired/topics/concept/troubleshoot-with-marvis-wired-config.html.

Team review note: Before submission, replace remaining highlighted TEAM TO INSERT placeholders and update planned-work statements only after fault injection and LLM diagnosis have been demonstrated. Do not convert planned work into past-tense claims until it has been demonstrated. Deterministic baseline forwarding and structured telemetry are already verified and should not be re-listed as unfinished. This note should be removed before final submission.